

Data Visualization

A Summary

Henri Heyden

stu240825

Kiel University

Winter Term 2025/26

Disclaimer

I do not guarantee this summary to contain every part of the lecture, as it is only supposed to summarize the things I personally found the most important when going through the slides.

This document was compiled on February 13, 2026.

Contents

1	Introduction	5
1.1	Why Visualize Data?	5
1.1.1	History of Visualization	5
2	Data	7
2.1	Dataset and Data Types	7
3	Perception	9
3.1	The Human Eye	9
3.1.1	Weber's Law & Fechner's Law	9
3.1.2	Three Stage Model	10
3.2	Gestalt Theory	10
4	Visual Encoding	13
4.1	Marks and Channels	13
4.1.1	Charts	13
5	Colour	15
5.1	Colour Palettes	15
5.1.1	Colour Vision Deficiency	16
5.1.2	Colour Spaces	16
6	Design	17
6.1	Graphical Integrity	17
6.2	Graphical Excellence	18
7	Tabular Data	19
7.1	Common Idioms for Tabular Data	19
7.2	Dimensionality Reduction	21
7.2.1	Principle Component Analysis	21
7.2.2	PCA Algorithm	22

7.2.3	Other Linear Dimensionality Reduction Techniques . . .	22
7.3	T-Distributed Stochastic Neighbour Embedding	23
7.4	Uniform Manifold Approximation and Projection for Dimension Reduction	23
8	Networks	25
8.1	Trees	26
9	Uncertainty	27
10	Spatial Data	29
A	Altair Guide	31
A.1	"My" Guide on Altair	31

Chapter 1

Introduction

Recent progress in hardware technology allowed computers to store an increasing amount of large data. A massive amount of data is produced and consumed each moment on the internet. In 2021, 2.5 quintillion bytes were created every day. A lot of this data is even publicly available, as reported by Archive.org in 2012, when they archived ten petabytes in total. An increase in data and surrounding importance makes it also important to teach about *correct* data visualization.

1.1 Why Visualize Data?

Data visualizations are vitally important to extract important information from often clouded data. Statistical measures such as mean, variance, and correlation are often not conclusive enough to do this job¹.

The human visual system is a comparatively high-bandwidth link at 10 million bits per second to the brain, therefore it can be used to convey a lot of information very fast.

1.1.1 History of Visualization

Data visualizations introduced different goals. At first, they were only used to convey already known information—the goal was to underline and present the facts. Then, they were also used to confirm or reject an already set hypothesis. This is called *confirmative analysis*. Finally, they are also used to find a hypothesis from interesting data. The latter is called *explorative analysis*.

The standard pipeline from data to visualization also changed. During the 1990s, this pipeline was straightforward: First data is filtered and then rendered. Recently, user interability became more important, leading to interactive diagrams, where each

¹<https://jumpingrivers.github.io/datasauRus/>

step from data to diagram can be modified in-time. Today, due to the popularity of machine learning algorithms such as transformers, a recent trend in visualization is also to use machine learning to visualize data. This method was conceptualized as machine learning models are usually hard to understand for a human. Therefore, a new goal is to use machine learning to create a model which solves the same task, but is humanly understandable.

Chapter 2

Data

Before we want to know how to visualize data, we may want to know about certain characteristics of data(sets) to exploit in a resulting diagram.

2.1 Dataset and Data Types

Generally, datasets are in the shape of three main types.

Tables Tables are (ordered) collections of rows which contain cells with individual values. Each dimension of a row is called an *attribute*.

Networks Networks or graphs, which includes trees, are collections of (valued) nodes or *vertices* and (valued) edges between nodes. Edges can be directed and undirected. A tree is a strict directed acyclic graph.

Spacial data Spacial data comes in the form of fields, which are continuous, and geometry, which is spacial. An example of continuous spacial data are meshes generated by an MRI scan or a vector field reporting wind speeds. Geometric spacial data are explicit spacial positions, such as state borders.

The values *themselves* in a dataset can have different kinds of attributes.

Categorical Categorical data can not be measured or counted in the form of numbers. Examples are texts, shapes, or items. This data type can be further split into two more distinct types.

Nominal Nominal data does not have an implicit order. Examples are eye colour, nationality, or gender.

Ordinal Ordinal data has some underlying unarithmic order. Examples are letter grades, education level, or sheet notes.

Quantitative Quantitative data can be expressed in numerical values, making it countable and including statistical data analysis. These kinds of data are also known as numerical data.

Again, this data type can be further split into two more distinct types.

Discrete Discrete data contains the values that fall under integers or whole numbers. The total number of students in a class is an example of discrete data. These data can't be broken into decimal or fraction values.

Continuous In contrary, continuous data is in the form of fractional numbers. It can e.g., be the height of a person, the length of an object, or income.

An important understanding behind each attribute is the semantics associated with it. To visualize data, one must keep the meaning behind attributes in mind. This also includes the occasional step to derive new attributes from given attributes, e.g., calculating the trade balance by subtracting the imports from exports.

Chapter 3

Perception

In this chapter we will look at important facts about human perception to consider when designing diagrams. An example that teaches this importance can be seen on https://www.csc2.ncsu.edu/faculty/healey/PP/index.html#jscript_search.

3.1 The Human Eye

Contrary to popular belief, the eye is not a perfect camera. Instead, it perceives limited colours at different angles *differently*. Additionally, the brain also has to interpret this information and fill in missing gaps. A similar conclusion can be made from the *Stroop effect*—where the difference between time needed recognizing meaning of colour and meaning of text delays understanding of colour. The distribution of cones on the retina makes outer angles appear less detailed than inner angles of the view field.

The so-called *ganglion cells* result in, what is called the *Hermann grid effect*, in which extreme difference in contrast can introduce hallucination. But in general, these cells enable us perceive contrast and brightness difference between similar colours, when they are directly between each other. Through them, we notice edges in gradients. A takeaway from this effect is, that local contrast around edges can be used to make it easier to detect edges. This is because the human eye perceives **relative differences** and not absolute differences.

3.1.1 Weber's Law & Fechner's Law

The quintessence of the last paragraph can be applied through Weber's and Fechner's laws. Weber's law states that for the notable perceived difference K , stimulus S , and stimulus difference dS we have $K \sim \frac{dS}{S}$. Fechner's Law states that for the perceived

brightness p , base stimulus S_0 , and sudden current stimulus S , we have $p \sim \ln \frac{S}{S_0}$.

3.1.2 Three Stage Model

The three stage model divides the processing of visual information into three main phases.

1. Rapid pre-attentive processing. Goal: Extract low-level properties of a sight.
2. Pattern perception. Goal: Recognizing objects and continuous contours.
3. Goal-directed processing. Driven by attention and linguistic connections.

In the first stage, the visual system focuses on the details that *pop out*. Using pre-attentive features as e.g., marks, exploits this idea. Pre-attentive features can be: Angle, length, closure, size, curvature, density, count, colour hue, colour intensity, intersection, termination, 3d depth cue, lighting direction, flicker, motion direction, motion velocity.

It has to be noted, that the overuse or random use of pre-attentive features can hinder this effect, and make it even more complicated to highlight e.g., points.

TODO: Maybe fill in missing content

3.2 Gestalt Theory

In Gestalt theory, the law of *Prägnanz* derives different laws based on how the mind groups shapes together to form objects. There are different laws based on this principle¹.

Good Law of Prägnanz Ambiguous and complex images are perceived simplified.

Law of Proximity Visually close things are related.

Law of Similarity Things that look alike are related.

Law of Connection Visually connected things are related.

Law of Closure Incomplete shapes are interpreted as complete.

¹To learn and differentiate between each law, I would recommend looking at the pictures in the slides and find external examples, e.g., https://en.wikipedia.org/wiki/Principles_of_grouping.

Law of Enclosure Enclosed shapes are related.

Law of Continuity Hidden objects are interpreted as fitting simple shapes.

Law of Symmetry Symmetric things are related.

Law of Past Experience Shapes that form known pictures are related.

Law of Figure and Ground Elements are either perceived as figures or background.

Law of Common Fate Elements moving in the same direction are related.

Chapter 4

Visual Encoding

Visual encoding is about how we visually abstract data to convey the most important informations to the human brain. This is formalized by defining *marks* and *channels*.

4.1 Marks and Channels

Marks are basic geometric elements that depict items or links. They can be e.g, points, lines, or areas. In contrast, channels, aka. dimensions or variables, control the appearance of marks. Channels can be *identity channels* and *magnitude channels*. Identity channels can be e.g., position, hue, or shape. Magnitude channels can be e.g., position, value, tilt, or size. Marks and channels are subjugated to different principles.

Expressiveness Principle The expressiveness principle states, that the visual encoding should express all of, and only, the information in the dataset attributes. This principle dictates that categorical attributes should map to identity channels, and that the ordered attributes should map to magnitude channels.

Effectiveness Principle The effectiveness principle states that the importance of the attribute should match the salience of the channel—its noticeability. The most important attributes should be encoded with the most effective channels.

Marks and channels together need the following component to form a visualization.

4.1.1 Charts

Charts are the *recipe* of a graph. It tells how to use channels and marks to visualize data. Different chart types offer different ways to represent the same data. When choosing channels in a chart, one can choose integral channels to highlight similarity or separable channels to highlight differences. It is vitally important to be careful when doing this, as using them both may make a graph very hard to read.

Chapter 5

Colour

As seen in the last "chapter", colour can be used as magnitude (value and saturation) and identity (hue) channels. When using colour for nominal or ordinal data, it is important to have a limited palette which either is noncontiguous or a finite gradient.

For quantitative data using hue can go wrong, but may work. When doing so, using the rainbow is a bad practice. A smarter solution is to use sequential colormaps such as viridis, magma, or limit the gradient to two main hues.

5.1 Colour Palettes

There are different kinds of colour palettes to choose from.

Categorical Categorical colour palettes aim for maximum distinguishability between individual colours.

Univariate Univariate colour palettes are for ordinal or quantitative data. They can be *sequential*, *diverging*, or *cyclic*. Sequential colour palettes change either hue, saturation, or value. Diverging colour palettes change two dimensions of colour, one steadily increasing and one firstly increasing and then decreasing. Cyclic colour palettes increase and then decrease at least one colour dimension.

Bivariate Bivariate colour palettes are univariate colour palettes which change at least two dimensions. A bivariate colour palette can e.g., change hue and saturation. Similarly to univariate colour palettes, bivariate colour palettes can also be sequential, diverging, or cyclic.

Binary Binary colour palettes are special bivariate colour palettes, where there are only two values for one of the two dimensions.

From these different kinds of palettes one must find the *right* choice depending on the underlying data.

5.1.1 Colour Vision Deficiency

Colour vision deficiency (CVD) is the decreased ability to see colour or differences in colour, or distinguish shades of colour. Colour palettes should be designed in mind of CVD, when many people see a graph. Overall, at least 4.5% of people of northern European decent have some kind of CVD. Safe colour palettes to use with CVD in mind are those which only use blue and orange or yellow. This maximises distinguishability for people with CVD.

5.1.2 Colour Spaces

There are different colour spaces to model colour perception. Red-Green-Blue (RGB) is useful for displays but very unuseful for information encoding. Instead often Hue-Saturation-Luminance (HSL) or Hue-Saturation-Value (HSV) is used to counteract this problem. Cyan-Magenta-Yellow-Key (CMYK) is used for printing.

Chapter 6

Design

Design is the backbone of a visualisation—it communicates how the chart uses (which) marks and channels to communicate the information. Good design may be aesthetic, useful, honest, and minimal¹.

Choosing a design can be a tough iterative process, that cycles between data-near analysis (what does this graph require?), and improvement (how can I make this graph look better?). The goal of this chapter is to formalize a critique of visualisation design. Edward Tufte defines two main goals for designs.

1. Practice: Graphical Integrity
2. Theory: Graphical Excellence

6.1 Graphical Integrity

Visual Representations of Data Must Tell the Truth

There are different ways to lie in a graph, one may offset a scale or omit it completely from the graph. One can also cleverly remove some data from the dataset to change the meaning of the chart.

To see through some of these distortions, Tufte developed the *lie factor*—a metric which compares the size of effect in the graph with the size of effect in the data. It is calculated in the following way—where S_{graph} is calculated by dividing the difference between minimal and maximal size in the graph by the minimal size.² S_{data} is analogously calculated with minimum and maximum of the data mapped to

¹No, I don't want to list *all* of Dieter Rams *Principles of Good Design*

²It is important, that the *size* in a graph is relative to the way the data is represented—is it length, area, volume, or something entirely else?

the graph.

$$LF := \frac{S_{\text{graph}}}{S_{\text{data}}}$$

6.2 Graphical Excellence

Tufte invented a method on how to achieve graphical excellence based on certain design principles.

Show Data Above all else, show the data.

Prioritise Data Ink Maximize the share of data ink³ within reason. *Within reason* because removing e.g., axis labels obfuscates the graph. The *data ink ratio* is calculated by dividing the amount of data ink by the total amount of ink in the graph. Ideally and naively 0.

Avoid Chartjunk Remove unnecessary visual elements in charts that distract the viewer from the information.

Erase Redundant Data Ink If some data ink is used multiple times for the same information, remove the duplicates.

Revise and Edit Again, having an iterative design process is important for good design.

Maximize Data Density Try to display a lot of data in a small area. The data density of a graphic is calculated by dividing the number of data entries displayed by the area of the graphic. A good strategy is to display multiple visually similar small graphs near each other.

³Ink used to display data

Chapter 7

Tabular Data

This chapter focuses on tabular data and which chart idioms to use when plotting it. As already mentioned, tabular items are identified by one or more keys and contain a vector of values specified by the attributes. Tabular items may also have no key, which is similar to sampled values.

7.1 Common Idioms for Tabular Data

Common idioms used for tabular data include the following.

Scatterplot A scatterplot expresses quantitative attributes. Channels, in this case horizontal and vertical position, are linked to the attributes. The mark used is commonly a point. Scatterplots can be used to visually express and find correlation (or lack thereof), clusters, outliers, or distribution. A scatterplot can be even more expressive by encoding additional attributes in other channels, such as colour, size, and shape.

Bar chart The bar chart can be used to express keyed items with commonly one quantitative attribute. The mark can be a line, while the channel is commonly the length of the line. The key can be ordered alphabetically or by attribute value. Bar charts can be used to compare values between keys.

Stacked Bar Chart A stacked bar chart is a variation of the bar chart, where an additional key is used to express different portions of the value plotted, e.g., consumption of food (key) by country (key). The additional categorical channel to differentiate between keys often is colour.

Streamgraph A streamgraph is a generalized stacked graph, where horizontal continuity is emphasized. One key is still channelled vertically while the other one may be continuous or a quantitative attribute.

Dot/Line Chart The dot and line charts are a form of a bar chart, where the marks are connected points. This can be practical, when the horizontal key is quantitative or when there is a relation between nearby keys.

Indexed Line Chart Indexed line charts are line charts with another key similar to stacked bar charts. This key is commonly encoded with colour as another line in the graph which does not have to be stacked on top.

Gantt Chart A Gantt chart uses lines as marks which are channelled by their horizontal length to express a duration. The horizontal attribute commonly is time but can be any continuous attribute. One key can be used to differentiate between different *tasks* by stacking them vertically.

Slopegraph A slopegraph uses two quantitative attributes, but only samples two points on the horizontal (often time) attribute. Each left and right sample is marked using points which are connected. The vertical attribute is associated with the key and shown on the left and right size of the two points. The change of the vertical attribute (positive or negative) is often also channelled using colour or/and size.

Parallel Coordinates parallel coordinate charts are slopegraphs with more than two points.

Heatmap For tables with two keys (can be continuous, e.g., space) and one quantitative attribute, heatmaps display values in a 2D matrix which is indexed by the two keys. Colour is used to channel the values. Heatmaps can be used to find clusters and outliers.

Cluster Heatmap A cluster heatmap is a heatmap, where the keys are ordered by identified clusters.

Radial Bar Chart, Star Plot Radial bar charts and star plots are radial bar charts.

Radar Plot A radar plot is a radial line chart. Should be avoided unless the categorical or quantitative key is radial.

Pie Chart, Coxcomb Chart Pie charts and coxcomb charts divide the area of a circle into different regions based on a categorical key. The size of each area is linked to the quantitative attribute to map. Similar to stacked bar charts, these charts can also be stacked to channel even another key. Small differences can be recognized easier when using coxcomb charts rather than pie charts.

Normalized Stacked Bar Chart this chart is a stacked bar chart, where the ratio of the second key is displayed by normalizing each stacked value to full vertical height. Each line is equivalent to one entire pie chart.

7.2 Dimensionality Reduction

A lot of more complex datasets can have too many dimensions to make them visually understandable by directly visualizing them. The goal of this section is to present different techniques, which reduce the dimensionality of the data set while keeping vital information.

7.2.1 Principle Component Analysis

Principle component analysis (PCA) is a dimensionality reduction techniques, which maps a set of $n \in \mathbb{N}$ -dimensional points to a space with $k < n$ dimensions, by eliminating *less variant* dimensions. We first have to go through some definitions.

Definition 7.2.1 (Variance and Covariance). For some vector $x \in \mathbb{R}^n$, we define the variance of the vector as the average square distance to the mean.

$$\text{Var}(x) := \frac{1}{n} \sum_{i \in [n]} (x_i - \bar{x})^2$$

The variance of some dataset is broadly the measure of how spread-out it is. The covariance is a similar measure, which instead looks at two vectors $x, y \in \mathbb{R}^n$, and declares, component-wise which dimensions of both vectors tend to change dependant on each other.

$$\text{Cov}(x, y) := \frac{1}{n} \sum_{i \in [n]} (x_i - \bar{x}) \cdot (y_i - \bar{y})$$

We have $\text{Var}(x) = \text{Cov}(x, x)$. Correlation is a similar measure as it is the covariance divided by the product of standard deviations.

Definition 7.2.2 (Eigenvectors and Eigenvalues). Eigenvectors and eigenvalues of some symmetric real matrix $A \in \mathbb{R}^{n \times n}$ are vectors v and scalars λ , which express $Av = \lambda v$. In other words, an eigenvector v only changes scale by λ , but not direction when it is transformed using A .

Eigenvalues of A are found by solving for the roots of the characteristic polynomial. I.e., find λ such that

$$0 = \det(A - \lambda I_n),$$

where I_n is the identity matrix with dimension n . Eigenvectors of A are then found by solving the following linear equation system for $v \in \mathbb{R}^n$.

$$(0)^n = (A - \lambda I_d) \times v$$

For symmetrical matrices, all eigenvectors should be orthogonal to each other.

7.2.2 PCA Algorithm

For some matrix $X \in \mathbb{R}^{d \times n}$ which contains points of dimension d in **row-wise** order, we perform PCA the following way.

1. Center¹ each point in X by the mean dimensional value.
2. Calculate the covariance matrix of the centered X . This can be done by filling each entry of a matrix $\in \mathbb{R}^{d \times d}$ according to the covariance formula. Alternatively, we can just calculate $X^T X$, as this matrix product is a product of the covariance matrix with some scalar.
3. Find the eigenvalues and eigenvectors of this covariance matrix according to Definition 7.2.2.
4. Order the eigenvectors by absolute value of the corresponding eigenvalue in decreasing order. The first/second/... vector is now the first/second/... principal component of X .
5. Pick the first $k < n$ principal components of X and put them **column-wise** in a matrix $V \in \mathbb{R}^{k \times d}$. Each vector can also be normalized, but this is not necessary to reduce the space.
6. Project pre-centered/normalized X to the k -dimensional subspace by multiplying $X \times V$.

Note: I omitted the closely-related subject of singular value decomposition (SVD), because the calculation of it is probably too complex for the exam. Just know, that it does something else (decompose a matrix) but in a similar way.

7.2.3 Other Linear Dimensionality Reduction Techniques

Independent Component Analysis (ICA) ICA is a computational method for separating a multivariate signal into additive subcomponents. This is done by assuming that at most one subcomponent is Gaussian and that the subcomponents are statistically independent from each other.

Linear Discriminant Analysis (LDA) LDA is a method to find a linear transformation of a multi-classed dataset, which separates each class in the best way. It finds the directions which maximize separation between classes and projects the data set according to these directions.

¹General standardization is optional.

7.3 T-Distributed Stochastic Neighbour Embedding

T-Distributed Stochastic Neighbour Embedding (t-SNE) is a statistical method for visualizing high-dimensional data by giving each point a location in 2D or 3D space.

The t-SNE algorithm comprises two main stages. First, t-SNE constructs a probability distribution over pairs of high-dimensional objects in such a way that similar objects are assigned a higher probability while dissimilar points are assigned a lower probability. Second, t-SNE defines a similar probability distribution over the points in the low-dimensional map, and it minimizes the Kullback–Leibler divergence² between the two distributions with respect to the locations of the points in the map.

Besides iteration count and learning rate, t-SNE uses another parameter, the *perplexity* σ —a hand-chosen parameter of t-SNE. As the authors state, “perplexity can be interpreted as a smooth measure of the effective number of neighbours [used for each point]. The performance of SNE is fairly robust to changes in the perplexity, and typical values are between 5 and 50.”

T-SNE is really useful when you know how to interpret it and how to set the parameters in a meaningful way. To use and interpret t-SNE effectively, choose the right perplexity, ignore the size of clusters and distances between them, differentiate between random noise and real meaning, and generate multiple plots.

7.4 Uniform Manifold Approximation and Projection for Dimension Reduction

Uniform Manifold Approximation and Projection for Dimension Reduction (UMAP) is a similar method as t-SNE, with the main difference being the inclusion of another parameter, *minimum distance*, which is the minimum distance between points in low-dimensional space. Instead of perplexity, the similar parameter is called *number of neighbours*, which is the number of approximate nearest neighbours used to construct an initial high-dimensional graph.

²https://en.wikipedia.org/wiki/Kullback%E2%80%93Leibler_divergence

Chapter 8

Networks

This chapter is about visualizing the network type of data. This includes graphs and trees. Common (visual) tasks on networks include topology based (finding properties of the graph structure), attribute based (finding specific nodes based on attribute values), browsing (finding paths between nodes), and estimation (temporal) tasks.

Most graphs can be drawn (embedded) in multiple ways, therefore we need to define some criteria for good layouts.

1. Minimize edge crossings and node overlap, distances between neighbouring nodes, drawing area, and edge bends.
2. Maximize angular distances between edges, aspect ratio disparities, and symmetry.

In many graphs these ideals contradict each other and introduce NP-hardness, therefore good layout is an optimization problem, which combines the different criteria into a weighted cost function. Multiple graph layouting idioms try to solve this problem.

Force-Directed Placement This algorithm iteratively pushes separate nodes apart from each other and pulls connected nodes together. This works well for small graphs or graphs where $|E| < 4|V|$.

Circular Layouts and Arc Diagrams In this layout, each nodes is positioned on a circle or line with bent edges. Nodes are ordered barycentric to minimize edge crossings.

Adjacency Matrix View To visualize a network, we can also derive an adjacency matrix and visualize this table. The resulting heatmap can again be clustered again by reordering the nodes.

Hierarchical Edge Bundling This is a step to do on any layout, which clusters edges with nearby source and destination nodes.

8.1 Trees

There are also visualization idioms specific to trees.

Indentation Place all items along vertically spaced rows. Use indentation to show parent/child relationship. All siblings and cousins of any degree are on the same indent.

Radial Node-Link Tree A radial node-link tree is an embedding of a tree, where the root node is placed in the center and all nodes on the same level are placed on circles with the center as midpoint. Children branch off in increased distance to the center.

Treemap A treemap is an embedding where each node is composed of some quantitative value and key, where the value of some parent is the sum of child values. The quantitative value of each nodes results in the rectangular area of the node embedding. Children are placed in the area of the parent.

Sunburst A sunburst chart is a hierarchical version of the pie chart.

Chapter 9

Uncertainty

In this chapter, the visualization of random variables and statistical data is discussed. Uncertain data is variant data. Uncertainty has to be visualized appropriately. The problem is, that many completely different data sources can have similar or identical plots of them. Therefore, identifying correlation between variables can not always be done by just comparing boxplots or directly plotting the data. The takeaway is, that the importance lies in visualizing mathematical confidence, not intuitive confidence.

Visualization can again be done using certain chart idioms.

Boxplot The boxplot visualizes a variant data source, by dividing the vertical part of the plot into different segments. Outliers are marked as dots and channeled using the appropriate quantitative value. Then the minimum is displayed as a horizontal line, which is connected by a vertical line to the 25th percentile. The 25th percentile starts a rectangle which has another horizontal line inside of it, which is the median. The rectangle ends with the 75th percentile, which is connected by a vertical line with the maximum. The 25th and 75th percentiles are calculated, by splitting the dataset in two according to the mean value of the whole set and calculating the lower and higher mean values.

Gradient and Violin Plot Gradient and violin plots are plots which encode the cumulative density of the area containing all values between minimum and maximum by transparency or width. For small datasets, these plots may confuse the reader and hallucinate meaning.

For bivariate data or temporal data, similar charts can be used.

Chapter 10

Spatial Data

IRRELEVANT FOR EXAM

Appendix A

Altair Guide

A good guide on how to use Altair to visualize Pandas `DataFrame` objects, can be found on https://altair-viz.github.io/user_guide/data.html.

Another great guide is found on <https://www.geeksforgeeks.org/python/introduction-to-altair-in-python/>.

A.1 "My" Guide on Altair

In Altair, we can plot Pandas `DataFrame` objects with `plt = altair.Chart(D).mark_m().encode(ch1=co1, ...)`—where D is a `DataFrame`, m is a mark, and $ch_1=co_1$ specifies which channels are expressing which attribute or column of the table.

Marks are specified with the methods listed in Table A.1. Possible channels are the parameters of the `encode` method listed in Table A.2. We can also hint which data type T a column co has by passing $co:T$ to some channel. T can be a value of Table A.3. Type hinting may be necessary because Altair needs to know how to interpret the data and determine e.g., scales from that. To combine and layer two generated charts, we can just write `plt1 + plt2`.

Mark	Method	Description
Point	<code>.mark_point()</code>	Scatter plot points
Circle	<code>.mark_circle()</code>	Like point but size-scaled
Line	<code>.mark_line()</code>	Line chart
Bar	<code>.mark_bar()</code>	Bar chart
Area	<code>.mark_area()</code>	Area chart
Tick	<code>.mark_tick()</code>	Small tick marks
Rect	<code>.mark_rect()</code>	Heatmaps
Rule	<code>.mark_rule()</code>	Reference lines
Text	<code>.mark_text()</code>	Labels
Square	<code>.mark_square()</code>	Square markers

Table A.1: Marks their methods in Altair.

Channel	Description
<code>x</code>	Horizontal position
<code>y</code>	Vertical position
<code>color</code>	Color of mark
<code>size</code>	Size of mark
<code>shape</code>	Shape of mark
<code>opacity</code>	Transparency
<code>tooltip</code>	Hover information
<code>row</code>	Facet by rows
<code>column</code>	Facet by columns
<code>detail</code>	Additional grouping
<code>order</code>	Drawing order
<code>text</code>	Text labels

Table A.2: Channels in Altair.

Type	Description
<code>Q</code>	Quantitative
<code>N</code>	Nominal
<code>O</code>	Ordinal
<code>T</code>	Temporal

Table A.3: Possible typehints for columns in Altair.